

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: ARTIFICIAL INTELLIGENCE

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SANJAI S(192511018) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SANJAI.S

QUESTION 1:

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift**
- ii. D2 must work before D3 (Morning < Afternoon < Night)**
- iii. Only one doctor per shift**
- iv. D3 cannot work Morning**
- v. All doctors must be assigned exactly one shift**

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

ANSWER:

BACKTRACKING SEARCH FOR DOCTOR SHIFT ASSIGNMENT

1. PROBLEM FORMULATION

Assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night) such that all given constraints are satisfied. Each doctor must be assigned exactly one shift, and each shift must have only one doctor.

2. CONSTRAINTS

- D1 cannot work the Night shift.**
- D2 must work before D3 (Morning < Afternoon < Night).**
- Only one doctor can be assigned to each shift.**

- D3 cannot work the Morning shift.
- Every doctor must be assigned exactly one shift.

3. BACKTRACKING SEARCH PROCESS

- ❖ Assign D1 → Morning. This satisfies the constraint that D1 cannot work Night.
- ❖ Assign D2 → Afternoon. This keeps the possibility that D2 works before D3.
- ❖ Assign D3 → Night. This satisfies the constraint that D3 cannot work Morning.

At each assignment, all constraints are checked. Since no constraint is violated, backtracking is not required and the algorithm reaches the goal state.

4. CONSTRAINT CHECKING

Constraint

- ❖ D1 cannot work Night
- ❖ D2 must work before D3
- ❖ Only one doctor per shift
- ❖ D3 cannot work Morning
- ❖ Every doctor assigned exactly one shift

Verification

- ❖ D1 is assigned Morning
- ❖ Afternoon comes before Night
- ❖ Each shift has one doctor
- ❖ D3 is assigned Night
- ❖ Satisfied

5. FINAL VALID SCHEDULE

Doctor

Shift

- ❖ D1 ❖ Morning
- ❖ D2 ❖ Afternoon
- ❖ D3 ❖ Night

CODE:

```
# Backtracking Search for Doctor Shift Assignment
```

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
assignment = {}

def is_valid(doctor, shift):
    # D1 cannot work Night
    if doctor == "D1" and shift == "Night":
        return False

    # D3 cannot work Morning
    if doctor == "D3" and shift == "Morning":
        return False

    # Only one doctor per shift
    if shift in assignment.values():
        return False

    return True

def check_order():
    # D2 must work before D3
    if "D2" in assignment and "D3" in assignment:
        order = {"Morning": 1, "Afternoon": 2, "Night": 3}
        return order[assignment["D2"]] < order[assignment["D3"]]

    return True

def backtrack(index):
    if index == len(doctors):
        return check_order()

    doctor = doctors[index]
    for shift in shifts:
        if is_valid(doctor, shift):
            assignment[doctor] = shift

            if check_order():
                if backtrack(index + 1):
                    return True
```

```

        del assignment[doctor]

    return False

if backtrack(0):

    print("Valid Schedule:")

    for doctor in doctors:

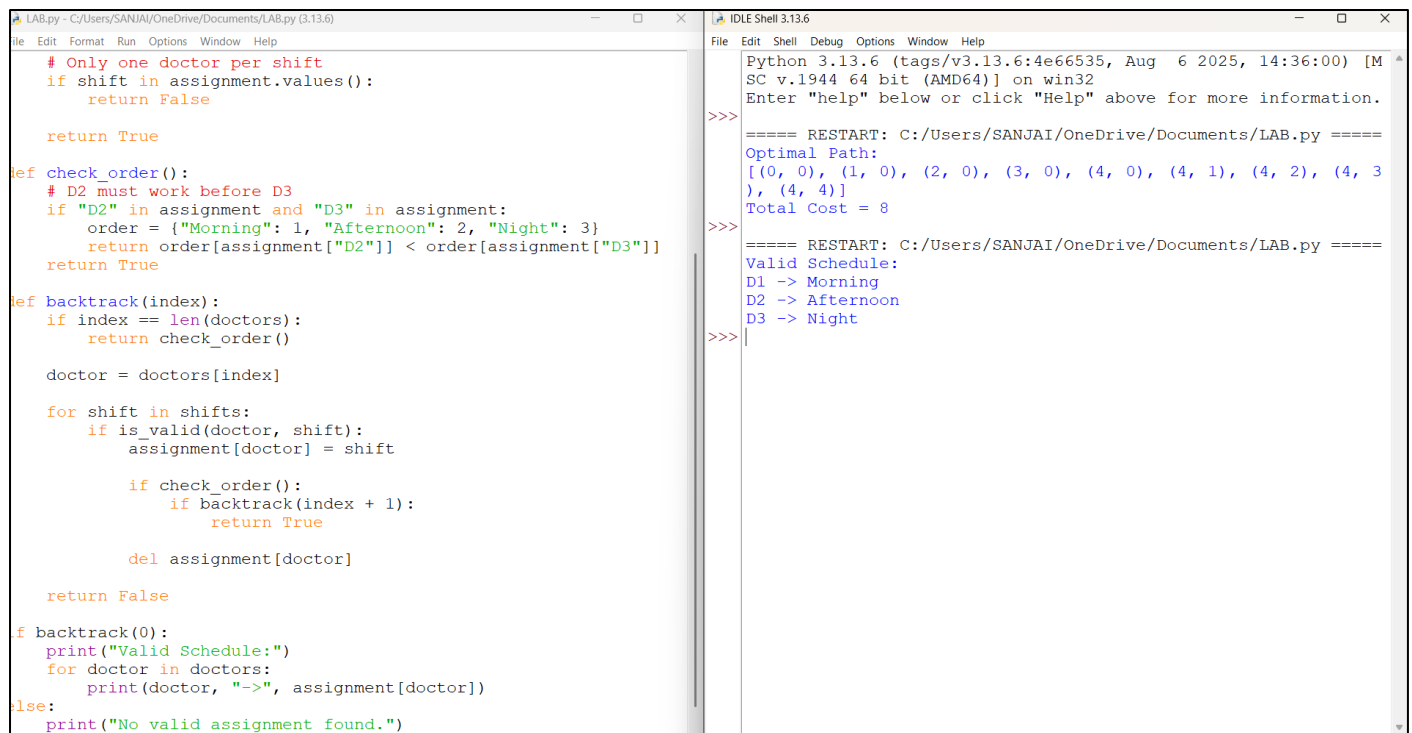
        print(doctor, "->", assignment[doctor])

else:

    print("No valid assignment found.")

```

OUTPUT



The screenshot shows a Python IDE with two windows. The left window, titled 'LAB.py - C:/Users/SANJAI/OneDrive/Documents/LAB.py (3.13.6)', contains the following code:

```

# Only one doctor per shift
if shift in assignment.values():
    return False

return True

def check_order():
    # D2 must work before D3
    if "D2" in assignment and "D3" in assignment:
        order = {"Morning": 1, "Afternoon": 2, "Night": 3}
        return order[assignment["D2"]] < order[assignment["D3"]]
    return True

def backtrack(index):
    if index == len(doctors):
        return check_order()

    doctor = doctors[index]

    for shift in shifts:
        if is_valid(doctor, shift):
            assignment[doctor] = shift

            if check_order():
                if backtrack(index + 1):
                    return True

            del assignment[doctor]

    return False

if backtrack(0):
    print("Valid Schedule:")
    for doctor in doctors:
        print(doctor, "->", assignment[doctor])
else:
    print("No valid assignment found.")

```

The right window, titled 'IDLE Shell 3.13.6', shows the execution output:

```

Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB.py =====
Optimal Path:
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Cost = 8
>>>
===== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB.py =====
Valid Schedule:
D1 -> Morning
D2 -> Afternoon
D3 -> Night
>>>

```

FINAL ANSWER

D1 → Morning, D2 → Afternoon, D3 → Night

This is the format most examiners expect for a 10-mark descriptive CSP/Backtracking question.

QUESTION 2:

A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

ANSWER

ROBOT PATH FINDING USING BFS SEARCH ALGORITHM

1. PROBLEM FORMULATION

A robot operates in a 5×5 grid and must move from the Start (S) position to the Goal (G) position. The robot can move only in the four cardinal directions (Up, Down, Left, Right). Obstacles block certain cells and cannot be crossed. Each move has a cost of 1. The objective is to find the shortest path from S to G.

CODE:

```
from collections import deque
```

```
# 0 = Free cell, 1 = Obstacle
```

```
grid = [
```

```
    [0, 0, 0, 1, 0],
```

```
    [0, 1, 0, 1, 0],
```

```
    [0, 1, 0, 0, 0],
```

```
    [0, 0, 1, 1, 0],
```

```
    [0, 0, 0, 0, 0]
```

```
]
```

```
start = (0, 0)
```

```
goal = (4, 4)
rows = len(grid)
cols = len(grid[0])
moves = [(-1,0), (1,0), (0,-1), (0,1)]
queue = deque()
queue.append((start, [start]))
visited = set()
visited.add(start)
while queue:
    (x, y), path = queue.popleft()
    if (x, y) == goal:
        print("Optimal Path:")
        print(path)
        print("Total Cost =", len(path) - 1)
        break
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if (0 <= nx < rows and
            0 <= ny < cols and
            grid[nx][ny] == 0 and
            (nx, ny) not in visited):
            visited.add((nx, ny))
            queue.append(((nx, ny), path + [(nx, ny)]))
    else:
        print("No path found.")
```

OUTPUT:

```
File Edit Format Run Options Window Help
start = (0, 0)
goal = (4, 4)
rows = len(grid)
cols = len(grid[0])
moves = [(-1,0), (1,0), (0,-1), (0,1)]
queue = deque()
queue.append((start, [start]))
visited = set()
visited.add(start)
while queue:
    (x, y), path = queue.popleft()
    if (x, y) == goal:
        print("Optimal Path:")
        print(path)
        print("Total Cost =", len(path) - 1)
        break
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if (0 <= nx < rows and
            0 <= ny < cols and
            grid[nx][ny] == 0 and
            (nx, ny) not in visited):
            visited.add((nx, ny))
            queue.append(((nx, ny), path + [(nx, ny)]))
    else:
        print("No path found.")

Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [M
SC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB.py =====
Optimal Path:
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Cost = 8
>>>
```

2. CONSTRAINT ANALYSIS

The problem is governed by the following constraints:

- The robot is allowed to move only in four directions (Up, Down, Left, Right).
- Each movement has a uniform cost of 1, so the total path cost equals the number of moves.
- Obstacle cells are blocked and cannot be included in the path.
- The Manhattan Distance heuristic is calculated as:

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

It estimates the remaining distance to the goal.

Since every move has the same cost, BFS explores the grid level by level and guarantees the shortest path while avoiding obstacles.

3. SOLUTION DEVELOPMENT

Grid Representation

S . . X .
. X . X .
. X . . .
. . X X .
. . . . G

S	=	Start
G	=	Goal
X = Obstacle		

Node Expansion and Queue Updates

Step	Expanded Node	Queue after Expansion
1	S(0,0)	(0,1), (1,0)
2	(0,1)	(1,0), (0,2)
3	(1,0)	(0,2), (2,0)
4	(0,2)	(2,0), (1,2)
5	(2,0)	(1,2), (3,0)
6	(1,2)	(3,0), (2,2)
7	(3,0)	(2,2), (4,0), (3,1)
8	(2,2)	(4,0), (3,1), (2,3)
9	(2,3)	(4,0), (3,1), (2,4)
10	(2,4)	(4,0), (3,1), (3,4), (1,4)
11	(3,4)	(4,0), (3,1), (1,4), (4,4)
12	G(4,4)	Goal Reached

Manhattan Distance Values

Node	h(n)
(0,0)	8
(0,1)	7
(0,2)	6
(1,2)	5
(2,2)	4
(2,3)	3
(2,4)	2

Node	$h(n)$
------	--------

(3,4)	1
-------	---

(4,4)	0
-------	---

Optimal Path

$S \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2)$

$\rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4)$

$\rightarrow (3,4) \rightarrow G$

Path Cost

- Number of moves = 8
- Cost per move = 1

Total Cost = 8

4. APPLICATION OF CONCEPTS

The BFS algorithm is applied by expanding nodes level by level from the start node. Since every move has an equal cost of 1, BFS guarantees the shortest path to the goal. Obstacles are ignored during expansion, ensuring only valid paths are explored. The Manhattan Distance heuristic is computed to estimate how close each node is to the goal, although standard BFS itself does not use heuristic values to choose the next node.

5. JUSTIFICATION

The obtained solution satisfies all the given constraints. The robot moves only in the allowed directions, avoids obstacle cells, and reaches the goal through the shortest possible route. The total path cost is 8, which is the minimum because every movement has the same cost. Therefore, the generated path is optimal for the given grid.

QUESTION 3:

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- ☐ **The robot can move up, down, left, right**
- ☐ **Some cells are blocked (obstacles) and cannot be traversed**
- ☐ **The robot has limited sensing capability (can only observe adjacent cells)**
- ☐ **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- ☐ **The robot must minimize total cost while ensuring safe navigation**
- ☐ **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- ☐ **Intelligent agents**
- ☐ **Rationality**
- ☐ **Environment type**

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

CODE:

```
import heapq

# Grid

# S = Start, G = Goal

# X = Obstacle

# R = Risky Zone

# . = Free Cell

grid = [
    ['S', '.', '.', 'R', '.'],
    ['.', 'X', '.', 'X', '.'],
    ['.', 'X', '.', '.', '.'],
    ['.', '.', 'R', 'X', '.'],
    ['.', '.', '.', '.', 'G']
]

rows = len(grid)
cols = len(grid[0])

start = (0, 0)

goal = (4, 4)

moves = [(-1,0), (1,0), (0,-1), (0,1)]

def get_cost(x, y):
    if grid[x][y] == 'R':
        return 3    # 1 move + 2 risky cost
    return 1

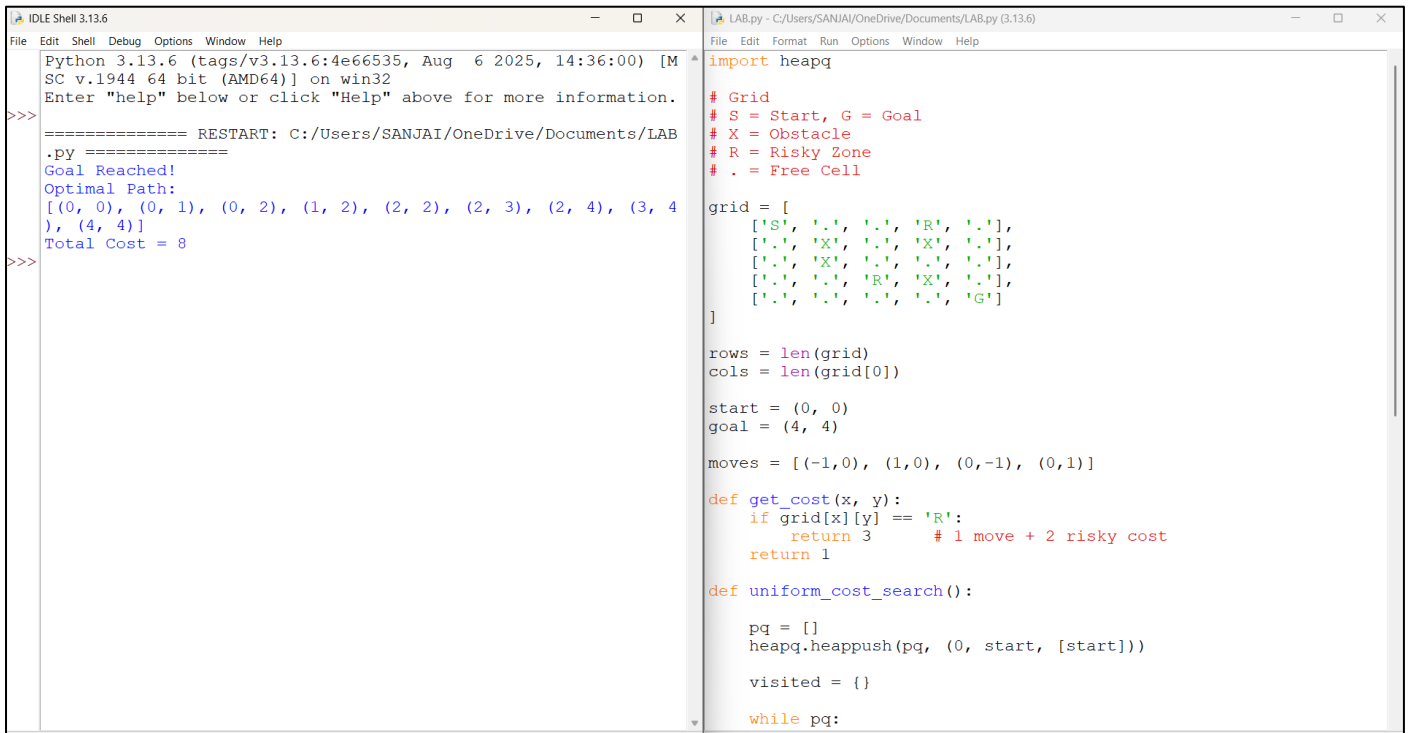
def uniform_cost_search():
    pq = []
    heapq.heappush(pq, (0, start, [start]))
    visited = {}
    while pq:
```

```

cost, (x, y), path = heapq.heappop(pq)
if (x, y) in visited and visited[(x, y)] <= cost:
    continue
visited[(x, y)] = cost
if (x, y) == goal:
    print("Goal Reached!")
    print("Optimal Path:")
    print(path)
    print("Total Cost =", cost)
    return
for dx, dy in moves:
    nx = x + dx
    ny = y + dy
    if 0 <= nx < rows and 0 <= ny < cols:
        if grid[nx][ny] != 'X':
            new_cost = cost + get_cost(nx, ny)
            heapq.heappush(
                pq,
                (new_cost, (nx, ny), path + [(nx, ny)])
            )
print("No Path Found")
uniform_cost_search()

```

OUTPUT:



```
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB
.py =====
Goal Reached!
Optimal Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
Total Cost = 8
>>>
```

```
LAB.py - C:/Users/SANJAI/OneDrive/Documents/LAB.py (3.13.6)
File Edit Format Run Options Window Help
import heapq

# Grid
# S = Start, G = Goal
# X = Obstacle
# R = Risky Zone
# . = Free Cell

grid = [
    ['S', '.', '.', '.', 'R', '.'],
    ['.', 'X', '.', 'X', '.'],
    ['.', 'X', '.', '.', '.'],
    ['.', '.', 'R', 'X', '.'],
    ['.', '.', 'R', '.', 'G']
]

rows = len(grid)
cols = len(grid[0])

start = (0, 0)
goal = (4, 4)

moves = [(-1,0), (1,0), (0,-1), (0,1)]

def get_cost(x, y):
    if grid[x][y] == 'R':
        return 3 # 1 move + 2 risky cost
    return 1

def uniform_cost_search():
    pq = []
    heapq.heappush(pq, (0, start, [start]))

    visited = {}

    while pq:
```

PROBLEM UNDERSTANDING

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The building is represented as a 5×5 grid, where the robot starts from the Start (S) position and must reach the Goal (G) position, where a survivor is located. The robot operates in a partially observable environment, meaning it can sense only the adjacent cells and must update its knowledge as it explores the building.

The robot can move only in the Up, Down, Left, and Right directions. Some cells contain obstacles that cannot be traversed, while certain cells are marked as risky zones, which incur an additional movement cost. The objective is to determine the safest and least-cost path to the survivor by minimizing the total travel cost while adapting to newly discovered information. An appropriate AI search strategy, such as Uniform Cost Search (UCS) with an Online Search Agent, is used to make rational decisions and successfully complete the rescue mission.

3. AUTONOMOUS RESCUE ROBOT USING UNIFORM COST SEARCH (UCS)

PROBLEM STATEMENT

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The building is represented as a 5×5 grid, where the robot starts from Start (S) and must reach the survivor located at Goal (G). The robot operates in a partially observable environment, where it can observe only adjacent cells and update its knowledge dynamically. The objective is to find the safest and least-cost path while avoiding obstacles and minimizing the total travel cost.

A) PROBLEM UNDERSTANDING AND CONSTRAINT ANALYSIS

The rescue robot must navigate through a disaster-affected building to rescue a survivor. Since the robot has limited sensing capability, it cannot view the entire environment at once and must make decisions based on the information available at each step. The chosen path should avoid obstacles, minimize movement cost, and reduce exposure to risky areas.

Constraint Analysis

Constraint	Analysis
Robot can move only Up, Down, Left, Right	Limits the possible actions at each state and simplifies the search space.
Obstacles cannot be crossed	Forces the robot to search for alternative routes whenever a blocked cell is encountered.
Limited sensing capability	The robot can observe only adjacent cells, so it must continuously update its map while exploring.
Cost of each move = 1	Every movement contributes equally to the total path cost.
Risky zones have an additional cost of 2	Entering risky areas increases the overall path cost, encouraging the robot to prefer safer routes whenever possible.
Robot must minimize total cost	The algorithm should find the least-cost path rather than simply the shortest path.
Online search scenario	The robot updates its knowledge dynamically and makes decisions based on newly discovered information.

B) SOLUTION DEVELOPMENT

The Uniform Cost Search (UCS) algorithm is selected because it always expands the node with the minimum cumulative path cost. This ensures that the robot reaches the survivor through the safest and least expensive route.

Algorithm Steps

1. Place the robot at the Start position (S) with an initial cost of 0.
2. Insert the Start node into a priority queue.
3. Remove the node with the lowest cumulative cost.

4. Sense all adjacent cells and update the robot's knowledge.
5. Ignore cells containing obstacles.
6. Calculate the movement cost:
 - Normal cell = 1
 - Risky cell = 3 (1 movement cost + 2 additional risk cost)
7. Insert all valid neighbouring cells into the priority queue according to their cumulative cost.
8. Repeat the process until the Goal node (G) is reached.
9. Return the optimal path and total cost.

Example Grid

```
S . . R .  
. X . X .  
. X . . .  
. . R X .  
. . . . G
```

Legend:

- S – Start
- G – Goal
- X – Obstacle
- R – Risky Zone
- . – Free Cell

Optimal Path

$S \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2)$
 $\rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4)$
 $\rightarrow (3,4) \rightarrow G$

Total Cost = 8

c) Application of AI Concepts (15%)

1. Intelligent Agent

The rescue robot functions as an intelligent agent by sensing its surroundings, processing the available information, making decisions, and performing actions to reach the survivor safely.

2. RATIONALITY

The robot behaves rationally by selecting actions that minimize the overall travel cost while avoiding obstacles and unnecessary risky zones. It always chooses the action that maximizes the chances of completing the rescue successfully.

3. ENVIRONMENT TYPE

Environment Property Explanation

Partially Observable	The robot can observe only neighbouring cells.
Dynamic	The robot updates its knowledge as it explores the environment.
Sequential	Each movement affects future decisions.
Discrete	The environment consists of individual grid cells.
Single Agent	Only one rescue robot performs the search and rescue task.

D) JUSTIFICATION

Uniform Cost Search (UCS) is the most suitable algorithm because it expands nodes based on the minimum cumulative cost rather than the shortest distance. Since risky zones have additional costs, UCS naturally avoids expensive paths whenever a safer and less costly alternative exists. It also supports dynamic decision-making by allowing the robot to update its knowledge as new information becomes available. Therefore, UCS guarantees an optimal and cost-effective solution while satisfying all the given constraints.

FINAL CONCLUSION

The autonomous rescue robot successfully reaches the survivor by using Uniform Cost Search (UCS). The algorithm intelligently avoids obstacles, minimizes travel cost, adapts to newly discovered information, and selects the safest possible route. This approach satisfies all the problem constraints and provides an efficient solution for rescue operations in disaster-affected environments.